

Characterization and Analysis of Deep Learning for 3D Point Cloud Analytics

Bongjoon Hyun^{ID}, Jiwon Lee, and Minsoo Rhu^{ID}

Abstract—A point cloud is a collection of points, which is measured by time-of-flight information from LiDAR sensors, forming geometrical representations of the surrounding environment. With the algorithmic success of deep learning networks, point clouds are not only used in traditional application domains like localization or HD map construction but also in a variety of avenues including object classification, 3D object detection, or semantic segmentation. While point cloud analytics are gaining significant traction in both academia and industry, the computer architecture community has only recently begun exploring this important problem space. In this paper, we conduct a detailed, end-to-end characterization on deep learning based point cloud analytics workload, root-causing the frontend data preparation stage as a significant performance limiter. Through our findings, we discuss possible future directions to motivate continued research in this emerging application domain.

Index Terms—Point cloud, data preparation, memory-intensive operation, characterization

1 INTRODUCTION

RECENTLY, we are witnessing an explosive adoption of *point clouds* (acquired by 3D sensors such as LiDARs and RGB-D cameras, Fig. 1) in numerous application domains such as autonomous driving [1], AR/VR [2], and graphics rendering [3]. A point cloud is an unordered set of points within the 3D Cartesian space, each point being uniquely identified by its $\langle x, y, z \rangle$ coordinates with an optional feature vector assigned per each point. As point clouds are a visual representation of a direct 3D measure of an object geometry or 3D scene, it provides rich geometric, shape, and scale information that complements 2D images, presenting an opportunity to better understand the surrounding environment. With the remarkable success of deep learning based approaches in computer vision applications, recent literature has successfully employed neural networks for *learning* features (rather than utilizing conventional, hand-crafted features [4]) for point clouds classification [5], [6], [7], object detection [8], and semantic segmentation [5], [6], [7], achieving state-of-the-art accuracy.¹

Despite the powerful and rich modality point cloud presents, the compute and memory access pattern of end-to-end point cloud analytics are fundamentally different than conventional convolutional neural networks (CNNs). Critically, unlike CNNs which spend most of their execution time on computationally intensive and regular convolution operations, point cloud analytics contain a diverse class of operators exhibiting highly sparse and irregular dataflow. To this end, the key objective of this study is to provide a detailed, end-to-end characterization on point cloud analytics, root-causing several crucial performance bottlenecks that prior work fails to properly address.

1. This paper focuses on point cloud algorithms that directly manipulate raw point clouds as alternative approaches (e.g., projecting 3D points into 2D [9], voxelization [10]) incur high memory consumption with low accuracy [11].

• The authors are with the School of Electrical Engineering, KAIST, Daejeon 34141, South Korea. E-mail: {bongjoon.hyun, jiwon.lee, mrhu}@kaist.ac.kr.

Manuscript received 26 May 2021; revised 11 July 2021; accepted 18 July 2021. Date of publication 26 July 2021; date of current version 16 Aug. 2021.

(Corresponding author: Minsoo Rhu.)

Digital Object Identifier no. 10.1109/LCA.2021.3099117

2 BACKGROUND AND MOTIVATION

2.1 End-to-End Point Cloud Processing Pipeline

Fig. 2 provides an overview of a typical point cloud processing pipeline. Below we detail each of its major processing stages.

Data Preparation. A raw point cloud is obtained by LiDAR sensors (e.g., 2.4 million points/second using a LiDAR with 128 channels [12]), which thereafter goes through the preprocessing procedure for data preparation. Point clouds are inherently noisy data structures because raindrops or particles floating in the air can reflect or refract laser beams radiated from the LiDAR sensor. Therefore, the data preparation stage enhances the quality of point clouds by removing such noisy points using *denoising filters* such as an SOR (statistical outlier removal) filter (Fig. 3a). SOR filter first finds k -nearest neighbors (k -NN) for each point in the point cloud. It then calculates both the mean and standard deviation of distances to k -NN for each point. Each mean and standard deviation of each point is compared with the global mean and standard deviation of the entire points within the point cloud. When the difference between the two exceeds a certain threshold, a subject point is considered noise. Other filters such as a voxel grid filter or RAN-SAC filter (the dotted-line stages in Fig. 2, used for downsizing the number of raw points or separating points belonging to the background such as the floor in an indoor scene, respectively) can optionally be included during the data preparation stage as appropriate, but the role of a denoising filter during data preparation is vital for high-quality downstream tasks (detailed further in Section 4.1). Once a denoised point cloud input is ready, the fixed point sampling algorithm is applied as means to choose a fixed number of points to feed into the subsequent stages of deep learning based 3D point cloud networks, e.g., (1024×3) in PointNet+ [6], which is similar to how the input image of the first layer of a CNN needs to be cropped at a fixed size, e.g., $(224 \times 224 \times 3)$.

Feature Extraction. As the raw point cloud is an unordered dataset, the objective of this stage is to extract meaningful features that encapsulate both the *global* and *local* structure of the point cloud set. Specifically, the feature extraction stage starts by first executing the *farthest point sampling* (FPS) algorithm to select a fixed number of *centroid* points to be utilized as the center of a group of point clouds. Fig. 3b illustrates an example where the FPS algorithm constructs N ($=3$) centroids that capture the *global* point cloud structure. The first centroid point is selected randomly and becomes part of the initial set of centroid points. Then on each iteration, the FPS algorithm computes the distances from every point within the point cloud to all the points within the centroid set. For each point, the distance to the current set of centroid points is derived as the minimum distance to the points within the current set of centroids. Using this distance information, FPS selects the point with the maximum distance as a new member to be included in the set of centroids. Once the centroid points are determined, we go through a k -NN or query ball point (QBP) algorithm to sample k neighboring points near the centroids, which effectively captures the *local* geometric shape of the point cloud. Finally, the sampled $(N \times (k + 1))$ points are processed by the MLP layers to extract the final feature vectors.

Downstream Task Processing. The backend stages of point cloud processing can vary dramatically depending on which downstream task the point cloud algorithm is designed for. For point cloud classification purposes, the backend stages are primarily structured using MLP layers whereas those for object detection might contain more complex operators such as RoI (region of interest) pooling or NMS (non-maximum suppression).

2.2 Key Challenges and Motivation

Prior work on Mesorasi [13] observed that the k -NN search algorithm followed by sparse vector gather operations incur significant

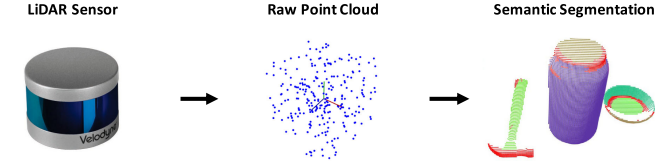


Fig. 1. Deep learning for 3D point cloud analytics applied on raw point cloud inputs collected by a LiDAR sensor.

performance overhead in point cloud analytics. This is because conventional dense-optimized ML accelerators such as Google TPUs are at odds with the memory-intensive k -NN or sparse and irregular vector gather operations. Consequently, Mesorasi proposes to augment the dense ML accelerator with a dedicated vector aggregation unit tailored for high-throughput vector gather operations.

While Mesorasi successfully addresses both the memory-intensive neighbor search and compute-intensive MLP stages of point clouds in a holistic fashion (the black-colored stages in Fig. 2), it overlooks the importance of the data preparation stage as the processing pipeline in Mesorasi assumes a *noise-free* input point cloud dataset. As we detail in Section 4.1, a point cloud processing pipeline *without* a denoising filter incurs an average 60.8 percent degradation in downstream task accuracy, underscoring the importance of the data preparation stage. Mesorasi also leaves the processing of FPS to general-purpose CPUs or GPUs, a stage that incurs non-negligible performance overhead (i.e., 1-13 percent of end-to-end execution time in our GPU evaluation, Fig. 5a). Overall, removing the irregularly scattered noisy points from an already sparse input point cloud is non-trivial, causing a hefty performance overhead. In the remainder of our study, we characterize the importance of data preparation, more specifically the denoising stage, and study its effect on end-to-end performance.

3 METHODOLOGY

Benchmarks. Table 1 summarizes the benchmarks used in our evaluation. We use PointNet++ [6] and PointTransformer [7] over the ModelNet40 dataset [14] for classification task. For semantic segmentation, we choose PointNet++ [6] on the Semantic3D dataset [15]. Finally, we choose PointRCNN [8] on the KITTI dataset [16] for object detection. For noise removal, we employ the well-known SOR filter, which all downstream tasks utilize for denoising. In all our evaluations, we do not include the voxel grid filter as well as the RANSAC filter as these filters are only optionally included under limited circumstances, e.g., when adjusting resolution or removing the floor in the background. Section 4.1 details how the noise-induced datasets are constructed.

Implementation. All the benchmarks used in our evaluation are constructed using TensorFlow v2.4.1, CUDA v11.0.221, cuDNN v7.6.5, and Thrust v1.9.09, which is primarily executed on NVIDIA's V100 GPU hosted by Intel Xeon E5-2698 v4 CPU. For those operators that are either not natively supported by TensorFlow and NVIDIA's cuDNN/cuBLAS, or not sufficiently optimized under publicly available implementations of our studied benchmarks, we heavily tune and optimize these CUDA kernels for a conservative analysis. In general, our custom-designed CUDA operators are on average $8.2\times$ (max $115\times$) faster than the

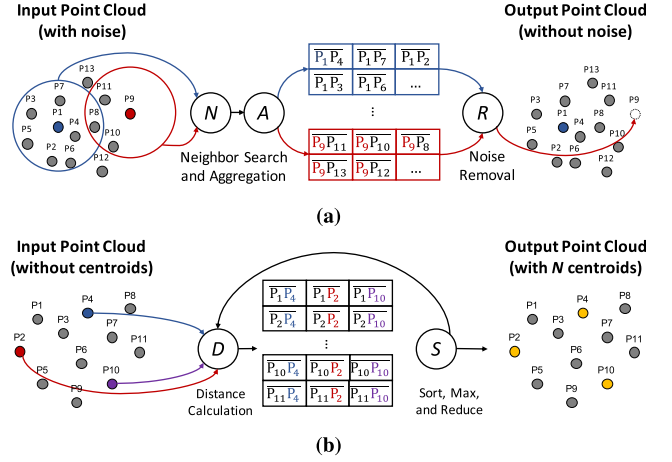


Fig. 3. Examples highlighting key primitives conducted for (a) SOR filter and (b) farthest point sampling (FPS). FPS is an iterative process where a single farthest point is determined within a single iteration of distance calculation/sort/max/reduce procedures (i.e., the three farthest points are extracted in $P4 \rightarrow P2 \rightarrow P10$ sequence).

TABLE 1
Evaluated Benchmarks and Datasets

Application domains	Algorithm	Dataset
Classification	PointNet++, PointTransformer	ModelNet40
Segmentation	PointNet++	Semantic 3D
Detection	PointRCNN	KITTI

publicly available open-source implementation of the studied point cloud analytics.

4 CHARACTERIZATION

4.1 Effect of Noise on Downstream Task Accuracy

To demonstrate the importance of filtering out noisy points from the point cloud input, we construct a noise-induced point cloud dataset by augmenting our training set with a synthetically generated noise model that properly accommodates the effect of real-world LiDAR sensor measurements. Our noise model incorporates two important sources of noise: the sensor measurement noise and raindrop noise. First, the LiDAR sensor measurement noise is generated based on a Gaussian distribution following prior literature [17]. The raindrop noise arises when particles floating in the air reflect or refract laser beams radiated from the LiDAR sensor [18]. Our raindrop noise model constructs a conical-shaped volume of a laser beam from the sensor to object, which is the shape of laser propagation if the laser beam propagates without being reflected or refracted by particles floating in the air (e.g., raindrops). Then, our model calculates the probability of the average number of raindrops residing in the conical-shaped volume whose distribution follows the Poisson distribution. With this derived probability distribution function of the average number of raindrops, our model conducts a Monte-Carlo simulation, generating raindrop noise, which can potentially block the laser beam to the

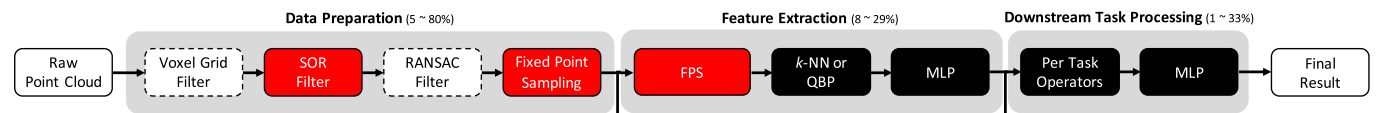


Fig. 2. A high-level overview of an end-to-end point cloud processing pipeline. The black-colored stages have been the primary target for acceleration in several related literatures [13]. As we detail in Section 4, the red-colored stages (the data preparation process followed by the FPS step) take up a significant fraction of execution time in an end-to-end point cloud inference, e.g., 5-80 percent of inference time for data preparation.

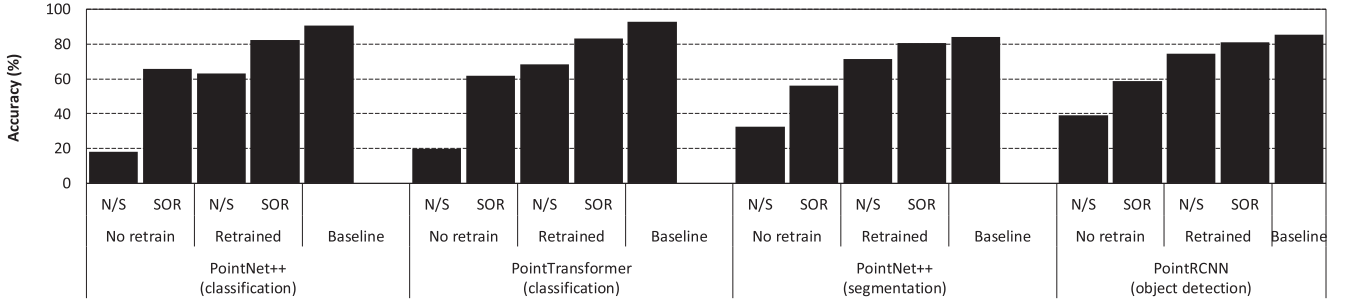


Fig. 4. Algorithmic performance of each downstream task depending on whether the SOR filter is included or not (SOR versus N/S) and whether the retraining phase with our noise-induced dataset is conducted or not (retrained versus no retrain). The baseline accuracy (the highest bar, Baseline) is measured when the vanilla noise-free dataset is used to both train and test the model.

object. Fig. 4 shows the accuracy of each downstream task, with and without the denoising SOR filter. We separate our analysis under two scenarios: 1) when each model is trained *only* with a noise-free, vanilla training dataset (No retrain), and 2) when the models are further *retrained* with the noise-augmented dataset (Retrained). Even with a 5 percent additional inclusion of noisy points, all three downstream tasks without an SOR filter (N/S) suffer from a significant drop in accuracy (average 19.0 and 60.8 percent degradation with and without retraining, respectively). However, when the inference pipeline employs the SOR filtering stage (SOR), each of the downstream tasks is able to recover back most of their loss in accuracy, achieving 92.7 percent of the baseline performance with retraining. Given that LiDARs are expected to be utilized under real-time environments where the raw point cloud input is highly likely to contain various noises, preprocessing the point clouds into a noise-free dataset is of vital importance.

4.2 Latency Breakdown

Our characterization in Section 4.1 confirmed the critical role denoising filters play in guaranteeing robust algorithmic performance. Therefore, this subsection seeks to identify major performance bottlenecks in an end-to-end point cloud processing pipeline that includes the SOR filtering stage. We show in Fig. 5a a breakdown of inference latency separated by *major stages* of data preparation, feature extraction, and downstream task processing. A breakdown of latency per *major compute operator* is also provided in Fig. 5b.

Breakdown Per Stage. With the exception of PointRCNN (object detection), all benchmarks consume a non-negligible fraction of their runtime on stages *outside* the feature extraction and downstream task processing, i.e., data preparation. Although the backbone network models of PointRCNN (such as region proposal network) are based out of PointNet++, other *non-deep* learning components (e.g., RoI pooling, NMS, others) consume significant processing cycles thus amortizing the overhead of SOR filters. Unfortunately, as for the other three benchmarks, the data preparation stage followed by FPS dominates the overall runtime and accounts for an average of 72.5 percent of latency. There are a couple of observations to be made from these results. First, even with a high-end V100 GPU, the bottleneck in data preparation renders the end-to-end processing throughput to “only” be at 10.5 frames-per-second even for classification, the fastest among the three downstream tasks (without the SOR filter, the sustained throughput becomes 53.9/23.6/0.8 frames-per-second for classification/segmentation/detection, respectively). Given point cloud analytics are deployed under real-time constraints (e.g., autonomous driving, robotics), carefully optimizing and accelerating the data preparation stage becomes vital, especially for mobile/edge systems that do not have the luxury of employing power-hungry, server-class GPUs. Another point to note is that with a dedicated accelerator for point clouds such as Mesorasi (which provides 3.6 $\times$ speedup

versus a high-end GPU for the red-colored stages in Fig. 2), the performance bottleneck of data preparation will become even more significant (i.e., Amdahl’s law), which calls out for system-level optimizations to address the limitations of frontend preprocessing steps.

Breakdown Per Operator. As we root-caused data preparation as a crucial performance limiter, gauging potential acceleration opportunities within this critical preprocessing stage is important. To this end, we show in Fig. 5b a breakdown of end-to-end inference time per major compute operators. As illustrated in Fig. 5a, the SOR filter and FPS cause the two most significant bottlenecks in point cloud analytics. Both of these stages heavily involve euclidean distance calculation as means to determine the relationship among each point within the point cloud (Fig. 3), exhibiting heavy use of sparse and irregular memory accesses accompanied by frequent sorting and gather operations. Consequently, for those downstream tasks dominated by the data preparation stage, we observe an average of 55.6, 1.43, and 15.3 percent of execution time spent conducting gather, distant calculation, and sorting, respectively. It is interesting to note that PointRCNN, which spends less than 6 percent of runtime on data preparation, still consumes significant compute cycles on gather and sorting operation, rendering these memory-limited operations a critical compute primitive in tackling the system-level bottlenecks of end-to-end point cloud inference.

4.3 Compute and Memory Efficiency

Overall, our analysis revealed that the unordered, sparse, and irregular nature of point cloud datasets naturally invoke a heavy

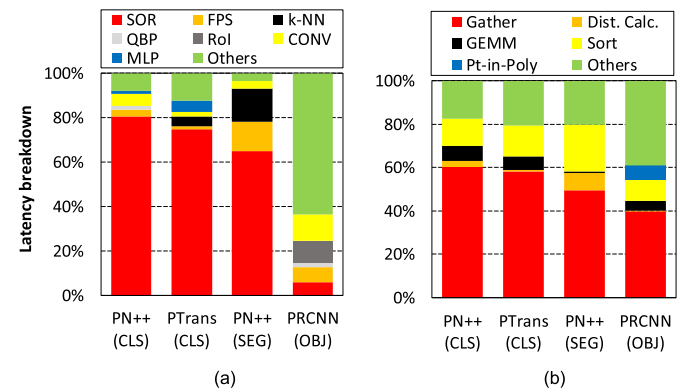


Fig. 5. Breakdown of inference time per (a) major stages within the point cloud processing pipeline, and (b) its major compute operators. PN++/PTrans/PRCNN refers to PointNet++/PointTransformer/PointRCNN models and CLS/SEG/OBJ designates the downstream task (corresponding to classification/segmentation/object detection) it is used for. The Point-in-Polygon (Pt-in-Poly) operator checks whether a given point is inside a given polygon or not (a polygon is a quadrangle for the object detection task since the shapes of bounding boxes are always a quadrangle).

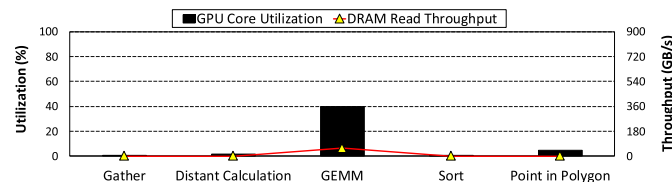


Fig. 6. The GPU core utilization (left) and DRAM throughput (right) when executing key compute operators. We utilize NVIDIA's *nvprof* for these measurements.

use of sparse memory accesses during various stages of point cloud processing. Because the computation applied on top of the sparsely (and irregularly) fetched scalar/vector data exhibit low compute intensity, it leads to highly inefficient utilization of both compute and memory throughput of GPUs. In Fig. 6, we show both GPU core utilization and DRAM read throughput during the execution of the major compute operators in Fig. 5b. A key observation we make is that the memory-intensive gather/distant calculation/sorting all exhibit very low memory bandwidth utility. This is because the average data fetch size of these operators (e.g., 24 bytes for distance calculation) are far less than the minimum data access granularity of the processor-memory interface, substantially wasting DRAM bandwidth. Such results imply that these compute primitives are memory latency limited, rather than memory throughput limited, providing valuable insights on optimizing their system-level bottlenecks.

5 FUTURE DIRECTIONS

Our characterization study in Section 4 revealed that the data preparation stage poses a critical system-level bottleneck in designing a robust and high-performance point cloud analytics pipeline. We observe several promising research directions for accelerating this critical step as follows. As discussed in Section 4.3, a significant fraction of point cloud processing is spent in sparse gather operations, distant calculation, or sorting operations, all of which exhibit highly memory latency limited behavior. One promising future direction of this work is to explore the efficacy of near-memory processing (NMP) solutions tailored for sparse gathers or sorting operations that helps reduce the data access latency of these primitives thereby improving system-level performance. A key challenge that is accompanied by an NMP based approach is how to partition and access the point cloud dataset (as well as the various intermediate data structures generated during the process of data preparation) across the memory channels that minimizes inter-channel communication, thereby maximizing the benefits of near-memory processing. We leave the development of accelerated computing solutions for data preparation as future work.

6 CONCLUSION

In this paper, we conduct an in-depth analysis of an end-to-end point cloud processing pipeline. Our analysis shows that the role of the data preparation stage is crucial in guaranteeing robust algorithmic performance, but the latency incurred can become prohibitive thereby causing a significant system-level bottleneck. To this end, we provide a detailed characterization on this crucial stage, breaking down the point cloud processing pipeline per major compute operators, uncovering the fact that a significant portion of execution is spent conducting highly memory-intensive gather/distant calculation/sorting operations. Future work is to develop accelerated computing solutions for end-to-end point cloud analytics using a near-memory processing based solution that best utilizes the key findings of our study.

ACKNOWLEDGMENTS

This work was supported in part by the research project supported by SK hynix Inc. under Grant G01210142-01, the Super Computer Development Leading Program of the National Research Foundation of Korea (NRF) funded by the Korean government (Ministry of Science and ICT(MSIT)) under Grant 2020M3H6A1085498, Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) under Grant 2020-0-01309, and Engineering Research Center Program through the National Research Foundation of Korea (NRF) funded by the Korean Government MSIT under Grant NRF-2018R1A5A1059921. We also appreciate the support from KAIST start-up package under Grant G04180036.

REFERENCES

- [1] B. A. Team, "Apollo: Open source autonomous driving," 2017. [Online]. Available: <https://github.com/ApolloAuto/apollo>
- [2] J. D. Stets, Y. Sun, W. Corning, and S. W. Greenwald, "Visualization and labeling of point clouds in virtual reality," in *Proc. SIGGRAPH Asia Posters*, 2017, Art. no. 31.
- [3] S. Rusinkiewicz and M. Levoy, "QSplat: A multi resolution point rendering system for large meshes," in *Proc. Annu. Conf. Comput. Graph. Interactive Techn.*, 2000, pp. 343–352.
- [4] R. B. Rusu, N. Blodow, and M. Beetz, "Fast point feature histograms (FPFH) for 3D registration," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2009, pp. 3212–3217.
- [5] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 77–85.
- [6] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "PointNet++: Deep hierarchical feature learning on point sets in a metric space," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 5105–5114.
- [7] H. Zhao, L. Jiang, J. Jia, P. Torr, and V. Koltun, "Point transformer," 2020, *arXiv:2012.09164*.
- [8] S. Shi, X. Wang, and H. Li, "PointRCNN: 3D object proposal generation and detection from point cloud," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 770–779.
- [9] Y. Liu, X. Huang, and Z. Zhang, "Learning to segment 3D point clouds in 2D image space," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 12252–12261.
- [10] T. Hinks, H. Carr, L. Truong-Hong, and D. F. Laefer, "Point cloud data conversion into solid models via point-based voxelization," *J. Surveying Eng.*, vol. 139, no. 2, pp. 72–83, 2013.
- [11] Z. Liu, H. Tang, Y. Lin, and S. Han, "Point-voxel CNN for efficient 3D deep learning," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 963–973.
- [12] Velodyne, "Alpha prime," 2019. [Online]. Available: <https://www.velodynelidar.com/products/alpha-prime/>
- [13] Y. Feng, B. Tian, T. Xu, P. Whatmough, and Y. Zhu, "Mesorasi: Architecture support for point cloud analytics via delayed-aggregation," in *Proc. 53rd Annu. IEEE/ACM Int. Symp. Microarchit.*, 2020, pp. 1037–1050.
- [14] Z. Wu et al., "3D ShapeNets: A deep representation for volumetric shapes," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 1912–1920.
- [15] T. Hackel, N. Savinov, L. Ladicky, J. D. Wegner, K. Schindler, and M. Pollefeys, "SEMANTIC3D.NET: A new large-scale point cloud classification benchmark," *ISPRS Ann. Photogrammetry Remote Sens. Spatial Inf. Sci.*, vol. IV-1-W1, pp. 91–98, 2017.
- [16] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2012, pp. 3354–3361.
- [17] V. Kurtz and H. Lin, "Kalman filtering with Gaussian processes measurement noise," 2019, *arXiv:1909.10582*.
- [18] J. V. Rivero, O. Schubert, H.-M. Kroll, and B. Buschardt, "A stochastic physical simulation framework to quantify the effect of rainfall on automotive lidar," *SAE Int. J. Advances Current Practices Mobility*, vol. 1, no. 2, 2019, Art. no. 8.

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.